

## Your Code

```
1 import time
2 import random
3 import math
4
5 def add(A, B):
6     n = len(A)
7     return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
8
9 def subtract(A, B):
10    n = len(A)
11    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]
12
13 def standard_multiply(A, B):
14    n = len(A)
15    C = [[0] * n for _ in range(n)]
16    for i in range(n):
17        for j in range(n):
18            for k in range(n):
19                C[i][j] += A[i][k] * B[k][j]
20    return C
21
22 def strassen(A, B):
23    n = len(A)
24    if n <= 32:
25        return standard_multiply(A, B)
26
27    mid = n // 2
```

## Input

stdin input

## Output

```
=====
Testing Size: 64
=====
IMAGE PROCESSING SYSTEM
-----
Original Matrix Size: 64 x 64
```

```

28
29     A11 = [row[:mid] for row in A[:mid]]
30     A12 = [row[mid:] for row in A[:mid]]
31     A21 = [row[:mid] for row in A[mid:]]
32     A22 = [row[mid:] for row in A[mid:]]
33
34     B11 = [row[:mid] for row in B[:mid]]
35     B12 = [row[mid:] for row in B[:mid]]
36     B21 = [row[:mid] for row in B[mid:]]
37     B22 = [row[mid:] for row in B[mid:]]
38
39     M1 = strassen(add(A11, A22), add(B11, B22))
40     M2 = strassen(add(A21, A22), B11)
41     M3 = strassen(A11, subtract(B12, B22))
42     M4 = strassen(A22, subtract(B21, B11))
43     M5 = strassen(add(A11, A12), B22)
44     M6 = strassen(subtract(A21, A11), add(B11, B12))
45     M7 = strassen(subtract(A12, A22), add(B21, B22))
46
47     C11 = add(subtract(add(M1, M4), M5), M7)
48     C12 = add(M3, M5)
49     C21 = add(M2, M4)
50     C22 = add(subtract(add(M1, M3), M2), M6)
51
52     C = []
53     for i in range(mid):
54         C.append(C11[i] + C12[i])
55     for i in range(mid):
56         C.append(C21[i] + C22[i])
57
58     return C
59
60 def run_experiment(n):
61
62     original_n = n
63     if (n & (n - 1)) != 0:

```

```

69     B = [[random.randint(1, 10) for _ in range(n)] for _ in range(n)]
70
71     start = time.perf_counter()
72     C1_full = standard_multiply(A, B)
73     standard_time = time.perf_counter() - start
74
75     start = time.perf_counter()
76     C2_full = strassen(A, B)
77     strassen_time = time.perf_counter() - start
78
79     match = True
80     for i in range(original_n):
81         if C1_full[i][:original_n] != C2_full[i][:original_n]:
82             match = False
83             break
84
85     print("IMAGE PROCESSING SYSTEM")
86     print("-----")
87     print(f"Original Matrix Size: {original_n} x {original_n}")
88     print(f"Processed Size: {n} x {n}")
89     print("\nStandard Matrix Multiplication")
90     print(f"Time: {round(standard_time, 6)} seconds")
91     print("\nStrassen Matrix Multiplication")
92     print(f"Time: {round(strassen_time, 6)} seconds")
93     print("\nVerification")
94     print("Both methods produce the same result." if match else "Results differ")
95     print("\nTime Complexity")
96     print("Standard :  $O(n^3)$ ")
97     print("Strassen :  $O(n^{2.807})$ ")
98
99 if __name__ == "__main__":
100
101     for size in [64, 128]:
102         print(f"\n{'='*30}\nTesting Size: {size}\n{'='*30}")
103         run_experiment(size)

```